

TABLE DES MATIÈRES

Codage & Algorithme de Huffman.....	4
But.....	4
Introduction	4
Exemple.....	4
Exemple.....	4
Définitions	5
Exemple :.....	5
Codes préfixes et arbres.....	6
Définition : code préfixe	6
Exemple.....	6
Définition : code complet.....	6
Exemple.....	6
Présentation du codage préfixe	7
Propositions	9
Proposition 1.....	9
Proposition 2.....	9
Exemples	9
Codage d'un texte.....	10
Exemple.....	10
Codage de longueur fixe	12
Exemple.....	12
Codage de longueur variable	13
Taille du fichier compressé	14

Codage de Huffman(1952)	15
Construction de l'arbre	15
Initialisation.....	15
Itération	15
Arrêt	15
Exemple.....	16
Choix de la représentation des données	18
Création de l'arbre Huffman.....	19
Principe.....	19
Calcul des fréquences et du nombre de lettres de fréquences non nulle	20
ExempleCodage des caractères fréquents	24
Création de la table de codage	26
ExempleCodage du texte	28
Décodage du message.....	29
Hypothèse.....	29
Principe.....	29
TasMin avec clés	32
Définition.....	32
Déclaration de la Structure Tas	32
Opérations	33
Fonction minTas(t) → TElement	33
Insertion dans un tas min	34
Principe.....	34
Procédure inserTasMin(e, t).....	35

Suppression du minimum	36
Principe.....	36
Procédure suppTasMin(t)	36

CODAGE & ALGORITHME DE HUFFMANN

But

Compression des données.

Coder les caractères d'un texte à l'aide de suites de 0 et de 1.

Introduction

Compression sans perte : où les données décompressées sont identiques aux données de départ.

Exemple

Compression de textes,

De données scientifiques

Ou du code compilé de programmes

Compression avec perte :

Exemple

Compression d'images et de sons : où une perte peut être acceptée si elle n'est pas perceptible, ou si elle est automatiquement corrigée par le récepteur.

Définitions

- Un alphabet A est un ensemble de lettres comme $\{0, 1\}$ ou $\{a, b, c, d\}$
- Un mot est une suite de lettres, comme 0110 ou abc
- La longueur d'un mot u est le nombre de lettres de u . $lg(u)=card(u)$
- Un mot p est préfixe (propre) d'un mot u s'il existe un mot v (non vide) tel que $u = pv$

Exemple :

Soit le mot $u = abcad$;

Les mots $p=\epsilon$, $p=ab$, $p=abca$ sont des préfixes propres de u .

En effet :

Il existe un mot $v=abcad$, $v=cad$, $v=d$

Tel que $u = p v$

Codes préfixes et arbres

Définition : code préfixe

Un ensemble P de mots non vides est un code préfixe si aucun mot de P n'est préfixe propre d'un autre mot de P .

Exemple

L'ensemble $P=\{0, 100, 101, 111, 1100, 1101\}$ est un code préfixe

Définition : code complet

Un code préfixe est complet si tout mot est préfixe d'un produit de mots de P .

C'est-à-dire, il admet au plus une décomposition comme produits de mots de P .

La décomposition, s'elle existe, s'obtient en lisant le mot de gauche à droite.

Exemple

10110101010111101100111 ce mot est produit de :

101 - 101 - 0 - 101 - 0 - 111 - 101 - 100 - 111

Présentation du codage préfixe

On utilise un arbre binaire dont les feuilles sont les lettres de l'alphabet A.

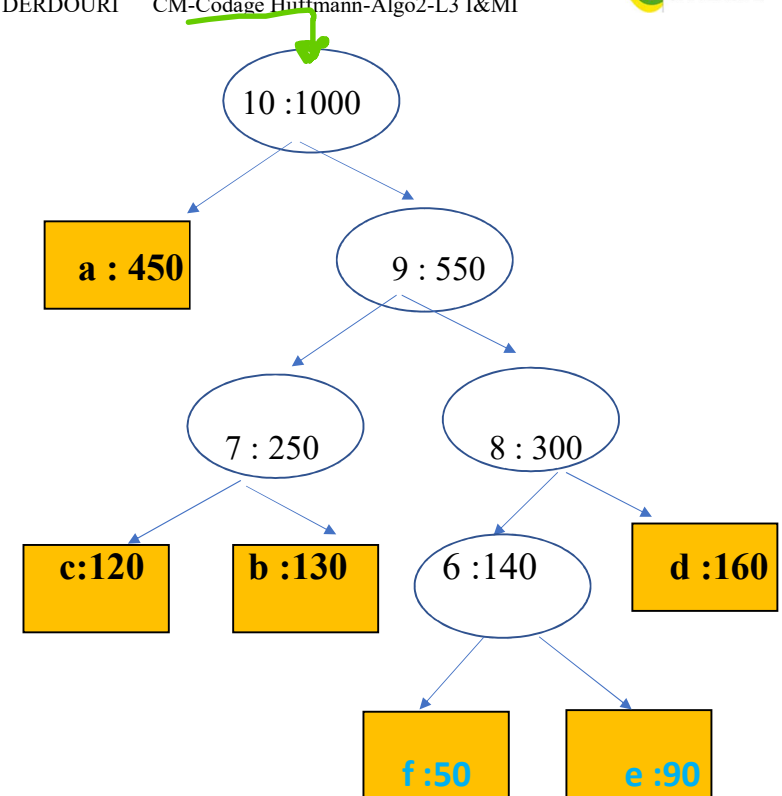
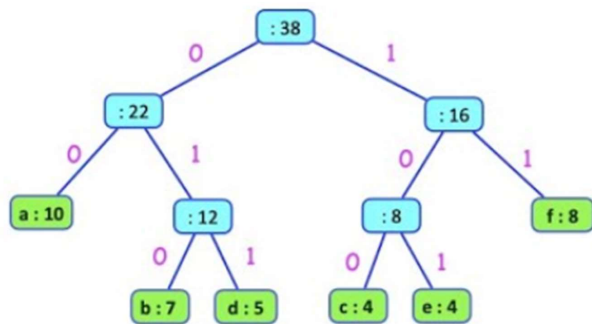
On interprète le mot de code binaire pour une lettre comme le chemin allant de la racine à cette lettre.

Où 0 signifie « bifurquer vers le fils gauche »

1 signifie « bifurquer vers le fils droit »

Chaque feuille est étiquetée avec un caractère et sa fréquence d'apparition dans le texte.

Chaque nœud interne est étiqueté avec la somme des poids des feuilles de son arbre.



Propositions

Proposition 1

Un ensemble fini de mots sur $\{0, 1\}$ est un code préfixe si et seulement si il est le code des feuilles d'un arbre binaire.

Rappel : un arbre binaire est dit complet si tous ses nœuds internes ont une arête 2 (càd : présence des deux fils pour chaque nœud interne).

Proposition 2

Un ensemble fini de mots sur $\{0, 1\}$ est un code préfixe complet si et seulement s'il est le code des feuilles d'un arbre binaire complet.

Examples

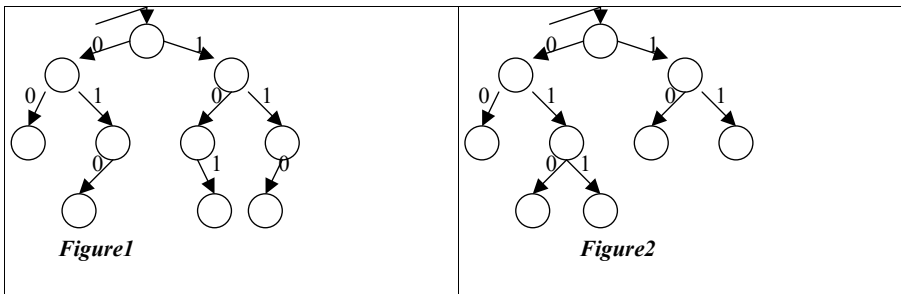


Figure1 : Codes des feuilles $P=\{00, 010, 101, 110\}$

Le code des feuilles est préfixé, mais il n'est pas complet.

Figure2 : Codes des feuilles $P=\{00, 010, 011, 10, 11\}$

P est un code préfixe complet.

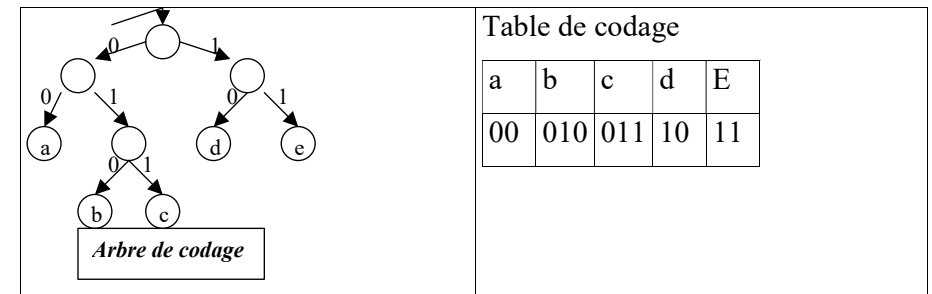
Codage d'un texte

Le codage d'un texte par un code préfixe (complet) se fait de la manière suivante :

- à chaque lettre est associé une feuille de l'arbre
- le code de la lettre est le code de la feuille.

Le codage du texte consiste simplement à remplacer chaque lettre par son code.

Example



Codage :

Chaque caractère du texte sera remplacé par son code.

Par exemple le mot aecdae devient 00.11.011.10.00.11

Décodage :

Il suffit de se faire guider par les étiquettes lors du parcours de l'arbre de codage.

Lorsque l'on est dans une feuille, on y trouve le caractère correspondant, et on recommence le décodage à partir de la racine.

Par exemple le code 00 010 011 correspond au mot abc.

Problème :

Le problème qui se pose est de minimiser la taille du texte codé.

Autrement dit, déterminer une meilleure table de codage qui minimise la taille du fichier compressé.

Codage de longueur fixe

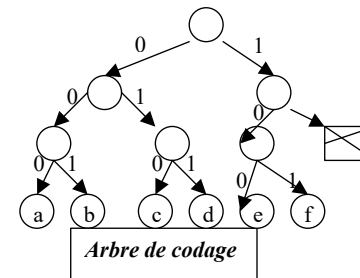
Exemple

Soit un fichier (texte) de 1000 caractères qui ne contient que les 6 caractères {a, b, c, d, e, f} de l'alphabet.

Fréquence d'apparition de chaque caractère dans le texte

a	b	c	d	e	f
450	130	120	160	90	50

Codage :



On a besoin de 3 bits pour représenter 6 caractères

a	b	c	d	e	f
000	001	010	011	100	101

→ il faut donc $3000 = 3 \times 1000$ bits pour coder entièrement le fichier.

Remarque

Le codage de longueur fixe est indépendant de la fréquence d'apparition des caractères dans le texte.

Codage de Huffman(1952)

Le code Huffman est un code à longueur variable qui exploite la répartition statistique des fréquences des caractères.

- Les caractères les plus fréquents sont représentés avec peu de bits
- Les caractères les moins fréquents avec plus de bits.
- Et aucun code n'est préfixe d'un autre. Cette condition est indispensable pour pouvoir décoder un texte codé sans ambiguïté.

Construction de l'arbre

L'algorithme de Huffman construit un arbre binaire complet qui donne un code optimal, en ce sens que la taille du code produit est minimale parmi la taille de tous les codes produits à l'aide de codes préfixes complets.

Initialisation

On construit une forêt d'arbres binaires formés chacun d'une seule feuille et triée selon les fréquences.

Chaque feuille correspond à une lettre du texte, et a pour valeur la fréquence d'apparition de la lettre dans le texte.

Itération

On fusionne deux des arbres dont la fréquence est minimale. Le nouvel arbre a pour fréquence la somme des fréquences de ses deux sous arbres.

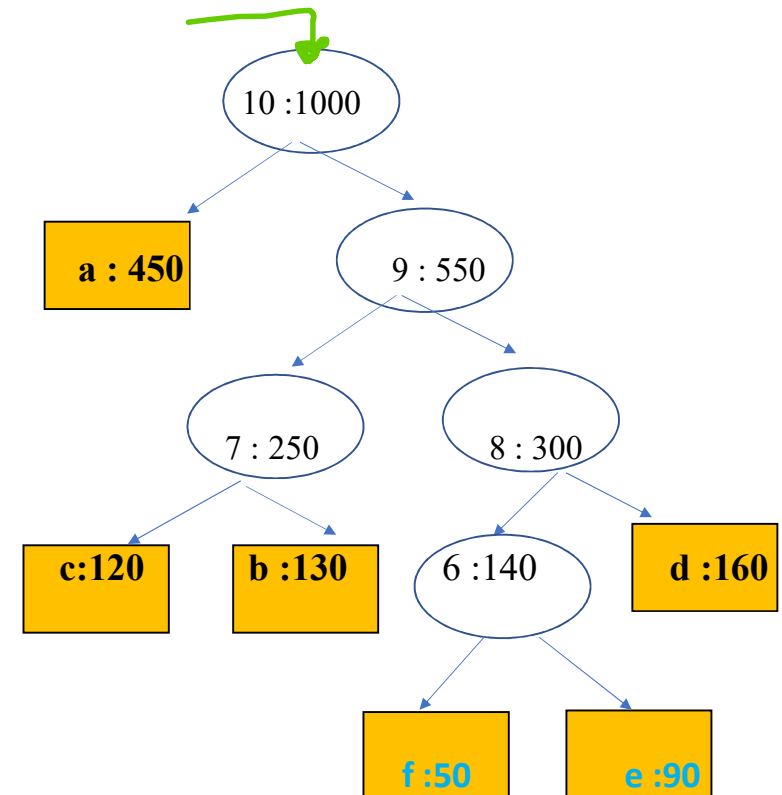
Remarque : à chaque itération, il faut maintenir la forêt triée selon la fréquence du nœud racine attaché à chaque arbre.

Arrêt

On termine quand il ne reste plus qu'un seul arbre qui est l'arbre résultat.

Exemple

Voir CM



Remarques

- ▣ Les caractères qui ont les plus petites fréquences se trouvent à la profondeur maximale de l'arbre.
- ▣ Chaque nœud interne à deux fils ($\rightarrow$ arbre de Huffman est complet).
- ▣ Si A est l'alphabet alors l'arbre de Huffman possède exactement $|A|$ feuilles (une pour chaque lettre) et $|A|-1$ nœuds internes.

Proposition

Si on suppose que n est le nombre de caractères de l'alphabet fréquent dans le texte. Alors l'arbre de Huffman est de taille $2n-1$.

En effet :

Comme il y a n feuilles dans l'arbres, il y a $(n-1)$ créations de nœuds internes.

Choix de la représentation des données

Si on suppose n est le nombre de caractères figurant dans le texte.

Alors l'arbre de Huffman est de taille $2n-1$.

En effet, comme il y a n feuilles dans l'arbre, il y a $(n-1)$ créations de nœuds internes).

On représente l'arbre de Huffman par un tableau Père où :

➤ Pour $m \geq 0$ et $p > 0$ on définit :

- $Pere[m] = p$ si m est fils droit de p
- $Pere[m] = -p$ si m est fils gauche de p
- $Pere[m] = 0$ si m est racine de l'arbre

- Pour chaque caractère c , on note $Frec[c]$ la fréquence d'apparition de c dans le texte.
- Seuls les caractères de fréquence non nulles sont représentés dans l'arbre.
- La création de l'arbre est contrôlée par un TasMin dont les clés sont les fréquences des caractères.

Création de l'arbre Huffman

Principe

- Les lettres de fréquence non nulles sont insérées dans le tas.
- On extrait du tas les caractères de fréquence minimale.
- Un nouvel arbre est créé, dont la fréquence est la somme des fréquences des deux arbres extraits, est inséré dans le tas.

Remarque

Comme il y a n feuilles dans l'arbre, il y a $(n-1)$ créations de nœuds internes.

Calcul des fréquences

A partir d'un message donné « Msg », calculer les fréquences de chaque lettre d'une part et d'autre part le nombre de lettres de fréquences non nulle

Hypothèse : on suppose que l'alphabet du texte à codé est $A=[a..z]$

Fonction calculFreq(nbCarAlph, Msg, freq) → Entier

// Renvoie le nombre de caractères de l'alphabet fréquents dans le message.

P.F nbCarAlph : entier (E)

msg : chaîne de caractères (E)

freq : tableau de fréquences (S)

Début

// Initialisation

Pour i variant de 0 à (nbCarAlph-1) faire

 Freq[i] ← 0

FPour

// Remplissage de la table des fréquences

codeA ← code du premier caractère de l'alphabet A

lgMSG ← Longueur(msg)

Pour i variant de 0 à (lgMSG-1) faire

 numCar ← code(msg[i])-codeA

 Incrémenter Freq[numCar] //rang du ième caractère de A

FPour

// Calcul du nombre de caractères de l'alphabet fréquents dans le message

nbCarAlphFreq ← 0

Pour i variant de 0 à (nbCarAlph-1) faire

 Si freq[i] > 0 alors

 Incrémenter nbCarAlphFreq

 Fsi

Fpour

Renvoyer nbCarAlphFreq

Fin

Rappel : La création de l'arbre est contrôlée par un TasMin dont les clés sont les fréquences d'apparition des caractères.

Procédure creatArbreHuffman(nbCarAlph, nbCarAlphFreq, Freq, Pere, filsG, filsD, Racine)

Début

//Phase Initialisation

// Allocation Mémoire et initialisation du Tas

tas ← AllocMemTas(nbCarAlphFreq)

tas ← initTas(tas)

// Les lettres de fréquence non nulle sont insérées dans le tas

Pour c variant de 0 de (nbCarAlph-1) faire

Si freq[c] > 0 alors

tas ← inserTasMin(c, tas, freq) //Un caractère de fréquence non nulle

Fsi

filsG[c] ← Indéfini //

filsD[c] ← Indéfini //

FPour

//Phase traitement

Construction un nœud interne de l'arbre :

On extrait du tas deux caractères de fréquence minimale.

Un nouvel arbre, dont la fréquence est la somme des fréquences des deux arbres extraits, est inséré dans le tas.

Pour c variant de nbCarAlph à (nbCarAlph+ nbCarAlphFreq -1) faire

c1 ← minTasMin(tas)

tas ← suppTasMin(tas, freq)

c2 ← minTasMin(tas, freq)

tas ← suppTasMin(tas, freq)

// Création d'un nouveau nœud

freq[c] ← freq[c1] + freq[c2]

pere[c1] ← -c

pere[c2] ← c

filsG[c] ← c1

filsD[c] ← c2

tas ← inserTasMin(c, tas, freq)

FPour

//Fin Traitement

Racine $\leftarrow$ minTasMin(tas)

tas $\leftarrow$ suppTasMin(tas, freq)

pere[Racine] $\leftarrow$ Indéfini // le nœud racine n'a pas de père

// Libération de l'espace Mémoire alloué par le tas

tas $\leftarrow$ libMem(tas)

Fin

Remarque

nœud racine = $(\text{nbCarAlphFreq} * 2 - 1) - 1$

= taille(arbre)-1

Le -1 car le tableau commence à l'indice 0.

Exemple à voir en CM

Codage des caractères fréquents

Rappel :

- Les feuilles de l'arbre représentent les caractères fréquents dans le texte.
- Chaque feuille est étiquetée avec un caractère plus sa fréquence d'apparition dans le texte.
- Chaque nœud interne est étiqueté avec la somme des poids de son fils gauche et de son fils droit.

Le calcul de la table de codage s'effectue pendant le parcours de l'arbre de Hoffmann représentée par le tableau père.

On interprète le mot de code binaire pour un caractère comme le chemin allant de la racine à la feuille représentant le caractère.

Où 0 signifie « bifurquer vers le fils gauche = $\text{pere}(\text{nd}) < 0$ »

1 signifie « bifurquer vers le fils droit = $\text{pere}(\text{nd}) > 0$ »

Pour retrouver le codage de chaque caractère il faut parcourir l'arbre de la racine jusqu'à la feuille qui contient ce caractère.

Chaque fois que l'on descend par une branche gauche, on ajoute un 0 au code, et un 1 quand on descend par la branche droite.

Le choix de la branche se fait en testant laquelle des deux branches conduit soit à un nœud comportant dans son ensemble le caractère cherché, soit à la feuille représentant ce caractère.

Création de la table de codage

Procédure CreaTableCodage(nbCarAlph, Freq, pere, tabCod)

Début

Pour c variant de 0 à (nbCarAlph-1) faire

Si $\text{freq}[c] > 0$ alors

$\text{tabCod}[c] \leftarrow \text{codage}(c, \text{pere})$

Fsi

Fpour

Fin

Fonction codage(nd, pere) → Chaîne composée d'une suite de 0 et de 1

Début

Si $\text{pere}[\text{nd}] = \text{INDEFINI}$ alors

Renvoyer chaîneVide

Sinon

Renvoyer $\text{codage}(|\text{pere}(\text{nd})|, \text{pere}) \oplus (\text{pere}[\text{nd}] < 0 ? \text{«}0\text{«} : \text{«}1\text{«})$

Fsi

Fin

Exemple à voir en CM

Codage du texte

Principe

On parcourt le texte tout entier et on remplace chaque caractère du texte son code.

Procédure CodageTexte(MsgSrc, tabCode, MsgCode)

P.F MsgSrc : chaîne (E)

tabCode : table de codage (E)

MsgCode : chaîne (S)

Début

MsgCode $\leftarrow$ ChaîneVide

longMsg $\leftarrow$ longueur(MsgSrc)

Pour i variant 0 à longMsg-1 faire

C $\leftarrow$ Msg[i]

MsgCode $\leftarrow$ MsgCode $\oplus$ tabCode[c]

Fpour

Fin

Décodage du message

Il suffit de se faire guider par les étiquettes lors du parcours de l'arbre de codage.

Lorsque l'on est dans une feuille, on y trouve le caractère correspondant, et on recommence le décodage à partir de la racine.

Hypothèse

Supposons que le message à décodé est représenté par un tableau de bits (suite de 0 et de 1).

Principe

On parcourt à l'aide de l'arbre de Huffman le message a décodé du début jusqu'à la fin.

Donc le décodage, effectue un parcours descendant de l'arbre :

- Les symboles successifs du texte codé : on suit la branche de gauche, si c'est un 0, celle de droite si c'est un 1 ;
- quand on arrive à une feuille de l'arbre, on a décodé un caractère du message source ;
- on remonte alors à la racine de l'arbre et on continue avec le symbole binaire suivant.

Hypothèse : on suppose que le message a décodé est correcte

Procédure decodageMessage(MsgCode, Racine, FG, FD, MsgSrc, Alphabet)

Début

lgMsgCode $\leftarrow$ longueur(MsgCode)

ind $\leftarrow$ 0

Erreur $\leftarrow$ faux

MsgSrc $\leftarrow$ ChaineVide

Tantque (ind < lgMsgCode **et non erreur**) faire

 decodage(ind, lgMsgCode, MsgCode, MsgSrc, Racine, FG, FD, Alphabet)

Fintantque

Fin

```
Procédure Decodage( ind, lgMsgCode, MsgCode, MsgSrc, Racine, FG, FD,
```

```
Alphabet)
```

```
Début
```

```
  Si (ind < lgMsgCode) alors
```

```
    Si estfeuille(racine, FG, FD) alors // ou racine < nbCarAlphFrec
```

```
      MsgSrc ← MsgSrc + Alphabet[racine]
```

```
    Sinon
```

```
      Si MsgCode[ind]='0' alors
```

```
        racine ← FG[racine]
```

```
      sinon
```

```
        racine ← FD[racine]
```

```
      fsi
```

```
      ind ← ind + 1
```

```
      Decodage(ind, lgMsgCode, MsgCode, MsgSrc,
```

```
        Racine, FG, FD, Alphabet)
```

```
    Fsi
```

```
  Fsi
```

```
Fin
```

Exemple Présence en CM

TasMin avec clés

Définition

Soit $\leq$ la fonction d'ordre.

Un **tasMin** est un arbre parfait qui vérifié la propriété suivante : le minimum est à la racine et le contenu de chaque nœud est inférieur ou égal au contenu de son fils gauche et de son fils droit.

Un **tas avec clés** gère des données avec priorités :

- Les données sont dans un tableau qui contient les caractères par exemple.
- Les priorités sont dans un deuxième tableau (les fréquences des caractères par exemple).

Déclaration de la Structure Tas

```
typedef char TElement ; // données
int * freq ; //priorité
Typedef struct{ //Tas
    int tailleMax ;
    int taille ;
    TElement * tab ;
} TasMin ;
```

Opérations

Fonction minTas(t) → TElement

Début

Renvoyer t.tab[0]

Fin

Insertion dans un tas min

Principe

L'insertion d'un nouvel élément se fait en deux temps :

- Placer le nouveau nœud à la première place libre en respectant la contrainte d'arbre parfait
- Permuter avec le parent jusqu'à l'obtention d'un tas ; c'est à dire on échange le contenu de la nouvelle feuille avec le contenu de son père, puis le contenu de celui-ci avec le contenu de son père et ainsi de suite jusqu'à ce que la propriété :

$\text{Freq}[\text{Contenu}(\text{pere}(e))] \leq \text{Freq}[\text{Contenu}(e)]$ est satisfaite.

Précondition : non estPleinTas(t)

Fonction inserTas(e, tas, freq) → Tas

P.F e : Telement (E) tas: Tas (E)

Début

n ← taille(tas) //n=place de l'insertion du nouveau élément

si n = 0 alors // (eq au tas vide) → On effectue l'insertion

t.tab[n] ← e

tas.taille ← taille(tas) + 1

sinon // on cherche le nœud d'insertion

p ← pere(n, t)

dp ← donneeAP(p, tas) // tas.tab[p]

Tantque (n > 0 et freq(dp) < freq(e)) faire // 0 = nœud racine

// on déplace le contenu du père vers le fils

tas.tab[n] ← dp

// le fils deviendra le père

n ← p

p ← pere(n)

dp ← donneeAP(p, tas)

fintq

// n = 0 ou freq(e) ≤ freq(dp)

// On effectue l'insertion

tas.tab[n] ← e

tas.taille ← taille(tas) + 1

fsi

Renvoyer tas

Fin

Suppression du minimum

Principe

- Placer le dernier nœud à la racine
- Supprimer le dernier nœud
- Echanger avec le plus petit fils jusqu'à l'obtention d'un tas

Algorithme de la suppression du maximum

Precondition : non estVideTas(t)

```
Fonction SuppMax(tas , freq) Tas
P.F tas : Tas (E)
Début
  r ← 0 //nœud racine
  // Placer le dernier nœud à la racine & suppression du dernier nœud
  dn ← taille(tas)-1
  tas.tab[r] ← donneeAP(dn, tas)
  // Supprimer le dernier nœud = décrémenter la taille du tas
  tas.taille ← dn
  si (non estVide(tas)) alors
    //Echanger le nœud courant (au départ le nœud racine) avec le plus grand fils
    jusqu'à l'obtention d'un tas
    e ← donneeAP(r, tas)
    verif ← faux
    fg ← filsGaucheAP(r, tas)
    tantque non verif et estExistNoeud(fg, tas) faire
      fd ← filsDroitAP(r, tas)
      fmax ← fg
      // Vérification de l'existence du fils droit & rech du fMax
      si estExistNoeud(fd,tas) et freq[donneeAP(fd,tas)]
        > freq[donneeAP(fg,tas)] alors
        fmax ← fd
      finsi
      si freq[e]>=freq[donneeAP(fmax,tas)] alors
        verif ←vrai
      sinon
        tas.tab[r] ← donneeAP(fmax, tas)
        r ← fmax
        fg ← filsGaucheAP(r, tas)
      finsi
    fintq
    //Placer la donnée de la feuille à la bonne place
    tas.tab[r] ← e
  Finsi
  Renvoyer tas
Fin
```